

Implementation, Validation, and Resource Analysis of a Regev-Style Quantum Factoring Pipeline

Technical Research Summary of the Qiskit Notebook

Summer Malik

July 2026

Abstract

This report documents the design, implementation, and testing of a parameterized Regev-style quantum factoring pipeline developed in Qiskit. The notebook constructs reversible modular-arithmetic primitives, composes them into controlled modular exponentiation, distributes the exponent across multiple input registers, applies a product quantum Fourier transform, parses measurements into multidimensional Fourier samples, and exposes those samples to a classical post-processing interface. End-to-end statevector-free simulations were executed with Qiskit Aer for $N = 15$ and $N = 21$, while larger instances were treated through circuit construction and logical resource estimation. The work establishes a functional quantum sampling front end and a useful experimental framework, but it does not yet implement Regev’s full lattice-based classical reconstruction. A coordinate-wise continued-fraction routine is included only as a small-instance diagnostic. Particular attention is therefore given to separating completed algorithmic components, empirical observations, implementation artifacts, and the remaining requirements for a faithful end-to-end Regev factorization experiment.

Scope statement. The notebook implements a Regev-inspired multidimensional quantum sampling circuit and its supporting reversible arithmetic. It should not yet be described as a complete implementation of Regev’s factoring algorithm because the lattice reconstruction stage is represented by an interface and placeholder rather than by a full reduction-and-recovery procedure.

Contents

1	Research Objective and Positioning	3
2	Principal Contributions	3
3	Mathematical Model	4
3.1	Parameter selection	4
3.2	Base family and early GCD detection	4
3.3	Prepared quantum state	4
3.4	Product Fourier transform	5
4	Reversible Arithmetic Stack	5
4.1	Controlled constant modular multiplier	5
4.2	Controlled modular multiplication and uncomputation	5
4.3	Modular exponentiation by repeated squaring	5
5	Software Architecture	6

6	End-to-End Execution Path	6
7	Classical Post-Processing Layers	8
7.1	Measurement-vector extraction	8
7.2	Toy continued-fraction diagnostic	8
7.3	Lattice interface	8
8	Experimental Environment and Methodology	8
9	Experimental Results	9
9.1	$N = 15$	9
9.2	$N = 21$	9
9.3	$N = 57$ arithmetic-construction test	10
9.4	$N = 2021$ construction and resource estimate	11
9.5	Logical resource scaling	14
10	Scientific Interpretation	14
10.1	What has been demonstrated	14
10.2	What has not yet been demonstrated	14
11	Limitations and Technical Risks	15
11.1	Missing lattice reconstruction	15
11.2	Toy post-processing is not Regev post-processing	15
11.3	Classical factor leakage during base generation	15
11.4	Base collisions modulo N	15
11.5	Forward/inverse QFT inconsistency	15
11.6	Measurement inconsistency in the exploratory large circuit	15
11.7	Resource estimate boundaries	16
11.8	Simulation and validation coverage	16
12	Recommended Next Research Phase	16
12.1	Phase I: arithmetic verification	16
12.2	Phase II: sampling verification	16
12.3	Phase III: full lattice post-processing	16
12.4	Phase IV: comparative and noise-aware study	17
13	Conclusion	17
A	Representative Core Logic	17
B	Reproducibility Checklist	18

1 Research Objective and Positioning

The project investigates how the quantum portion of Regev’s factoring framework can be expressed, tested, and scaled in a modern circuit-construction environment. Regev’s proposal replaces the single long exponent register characteristic of textbook presentations of Shor’s algorithm with a higher-dimensional sampling structure and shifts additional work into polynomial-time classical post-processing [1]. The notebook focuses on the engineering questions that arise before the asymptotic claim can be evaluated experimentally:

- How should the input dimension and per-register precision be selected from the bit length of N ?
- How can several exponent registers coherently control modular multiplication into one shared work register?
- How should the product Fourier transform and measurement layout be represented in Qiskit?
- How can raw Qiskit bitstrings be converted into mathematically meaningful vectors?
- What resource growth appears after decomposition into simulator-supported gates?
- Which parts of a complete Regev implementation remain absent from the current notebook?

The result is best characterized as an *implementation and validation scaffold*: it connects reversible arithmetic, circuit generation, simulation, measurement interpretation, and preliminary classical diagnostics in one input-driven workflow.

2 Principal Contributions

The notebook makes seven concrete contributions.

1. **Reusable reversible arithmetic.** It defines controlled constant modular multiplication and controlled modular exponentiation using imported Häner-style modular-addition components, controlled swaps, modular inverses, and explicit uncomputation.
2. **Multidimensional exponent architecture.** It constructs d independent exponent registers that share a modular output register and an auxiliary register.
3. **Input-driven parameterization.** It derives n , d , the per-dimension register size n_d , the Fourier modulus M , and a logical qubit estimate directly from N .
4. **End-to-end simulation pipeline.** It validates the input, generates bases, builds the circuit, transpiles it, runs Qiskit Aer, and summarizes the resulting Fourier samples.
5. **Endian-safe measurement decoding.** It converts Qiskit’s displayed bitstrings into ordered vectors in $\{0, \dots, M - 1\}^d$.
6. **Two classical interfaces.** It provides both a formally labeled lattice-post-processing placeholder and a separate continued-fraction/order heuristic for toy instances.
7. **Resource and scaling diagnostics.** It records logical qubit counts, circuit depth, and decomposed gate counts for representative composite inputs.

3 Mathematical Model

3.1 Parameter selection

For an odd composite integer N , the notebook defines

$$n = \text{bitlength}(N), \quad d = \lceil \sqrt{n} \rceil. \quad (1)$$

Two choices are implemented for the number of qubits in each exponent register:

$$n_d^{(\text{cover})} = \left\lceil \frac{2n}{d} \right\rceil, \quad (2)$$

$$n_d^{(\text{notebook})} = \left\lceil \frac{n}{d} + d \right\rceil. \quad (3)$$

The primary experiments use the first mode, which ensures

$$dn_d \geq 2n. \quad (4)$$

Each exponent coordinate ranges over

$$x_i \in \{0, \dots, M-1\}, \quad M = 2^{n_d}. \quad (5)$$

The logical register allocation is

$$\underbrace{dn_d}_{\text{exponent registers}} + \underbrace{n}_{\text{shared output}} + \underbrace{(n+1)}_{\text{auxiliary workspace}}, \quad (6)$$

so the notebook's estimated total is

$$Q_{\text{logical}} = dn_d + 2n + 1. \quad (7)$$

This is a register-level estimate for the implemented construction, not a fault-tolerant physical-qubit estimate.

3.2 Base family and early GCD detection

The circuit uses d squared-prime bases

$$a_i = p_i^2, \quad (8)$$

subject to $\gcd(a_i, N) = 1$. Before accepting a base, the code computes $\gcd(p_i, N)$. If a candidate prime divides N , the factor is recorded immediately and that prime is skipped.

This is mathematically valid as an early-exit optimization, but it has an important consequence for interpretation: for $N = 15$, $N = 21$, and $N = 57$, the base-generation routine itself encounters a nontrivial divisor. Therefore, those factors are known classically before the quantum sample is processed. The small-instance demonstrations validate circuit behavior, register wiring, and post-processing interfaces; they do not independently demonstrate quantum-assisted factor discovery.

3.3 Prepared quantum state

After Hadamard gates are applied to all exponent registers and the shared output register is initialized to $|1\rangle$, the intended modular-exponentiation action is

$$\frac{1}{\sqrt{M^d}} \sum_{\mathbf{x} \in \{0, \dots, M-1\}^d} |x_1, \dots, x_d\rangle |1\rangle |0\rangle \mapsto \frac{1}{\sqrt{M^d}} \sum_{\mathbf{x}} |\mathbf{x}\rangle \left| \prod_{i=1}^d a_i^{x_i} \bmod N \right\rangle |0\rangle. \quad (9)$$

The shared auxiliary register is intended to return to zero through reversible uncomputation.

3.4 Product Fourier transform

The multidimensional Fourier layer is implemented as a tensor product of one-dimensional transforms,

$$\mathcal{F}_M^{\otimes d} = \underbrace{\mathcal{F}_M \otimes \cdots \otimes \mathcal{F}_M}_{d \text{ factors}}. \quad (10)$$

The principal simulation pipeline uses the inverse QFT on each exponent register, matching the usual phase-estimation convention. An earlier helper named `multidim_qft` applies the forward QFT; the distinction is discussed in [section 11](#).

4 Reversible Arithmetic Stack

4.1 Controlled constant modular multiplier

The function `controlled_constant_modulo_multiplier(constant, N, n)` allocates registers for a control qubit, an n -qubit input, an n -qubit accumulator, and a one-qubit flag. For each input bit i , it computes

$$c_i = 2^i \cdot \text{constant} \bmod N \quad (11)$$

and invokes a double-controlled constant modular adder. The reverse iteration over bit positions is inherited from the arithmetic construction used by the cloned implementation repository.

4.2 Controlled modular multiplication and uncomputation

The controlled modular multiplication gate performs three steps:

1. apply the controlled constant modular multiplier;
2. conditionally swap the primary and auxiliary n -qubit registers;
3. compute the modular inverse $a^{-1} \bmod N$ and apply the inverse multiplier to clear temporary information.

This structure is designed to realize a clean reversible map while returning workspace to its initial state.

4.3 Modular exponentiation by repeated squaring

For exponent bit i , the implementation precomputes

$$a_i^{(\text{partial})} = a^{2^i} \bmod N \quad (12)$$

and appends a controlled modular multiplication using that constant. Consequently, a q_d -qubit exponent register controls q_d modular-multiplication blocks. The interface is

$$\text{modular_exponentiation_gate}(a, N, n, q_d). \quad (13)$$

The notebook's later input-driven pipeline passes $q_d = n_d$, eliminating the fixed $2n$ exponent-register assumption present in many Shor-oriented implementations.

5 Software Architecture

Function or component	Role in the pipeline
<code>validate_factor_input</code>	Rejects $N \leq 1$ and prime inputs; returns the trivial factor for even N ; permits odd composite inputs.
<code>regev_parameters</code>	Computes n, d, n_d, M , total exponent qubits, and the register-level total in equation (7).
<code>generate_regev_bases</code>	Produces coprime squared-prime bases and records nontrivial factors encountered by GCD checks.
<code>controlled_constant_modulo_multiplier</code>	Builds a controlled modular multiplier from modular-addition primitives.
<code>controlled_modular_multiplication_gate</code>	Adds swaps, inverse multiplication, and uncomputation around the multiplier.
<code>modular_exponentiation_gate</code>	Applies repeated-squaring constants controlled by exponent bits.
<code>build_multidim_qft_on_x</code>	Applies a forward or inverse QFT independently to each exponent register.
<code>build_regev_simulation_circuit</code>	Allocates registers, prepares superposition, performs modular exponentiation, applies the product QFT, and measures all exponent qubits.
<code>bitstring_to_regev_vector</code>	Reverses Qiskit's displayed bit order, partitions the result by register, and returns a d -tuple of integers.
<code>summarize_regev_counts</code>	Aggregates measured vectors, counts, probabilities, and raw bitstrings.
<code>run_regev_simulation</code>	Builds, transpiles, simulates, reports resources, and returns the circuit, metadata, counts, and summarized samples.
<code>toy_factor_from_regev_summary</code>	Runs coordinate-wise rational reconstruction and standard order-to-factor GCD tests for toy diagnostics.
<code>lattice_postprocessing_placeholder</code>	Defines the interface through which multidimensional samples will later enter a true lattice reconstruction routine.
<code>estimate_regev_resources</code>	Reports logical register requirements without attempting large simulations.

6 End-to-End Execution Path

For a valid odd composite input, the main pipeline follows the sequence below.

1. Compute (n, d, n_d, M) and allocate d exponent registers, one n -qubit output register, one $(n + 1)$ -qubit auxiliary register, and dn_d classical bits.
2. Apply $H^{\otimes n_d}$ to each exponent register.
3. Initialize the shared output register to the binary encoding of one.
4. Sequentially append one modular-exponentiation gate per dimension, all acting on the same output and auxiliary registers.
5. Apply an inverse QFT to each exponent register.

6. Measure all dn_d exponent qubits.
7. Transpile with optimization level 1 for `AerSimulator`.
8. Execute a specified number of shots and decode each bitstring into a vector $\mathbf{z} \in \{0, \dots, M-1\}^d$.
9. Return the quantum samples either to the diagnostic continued-fraction routine or to the future lattice reconstruction layer.

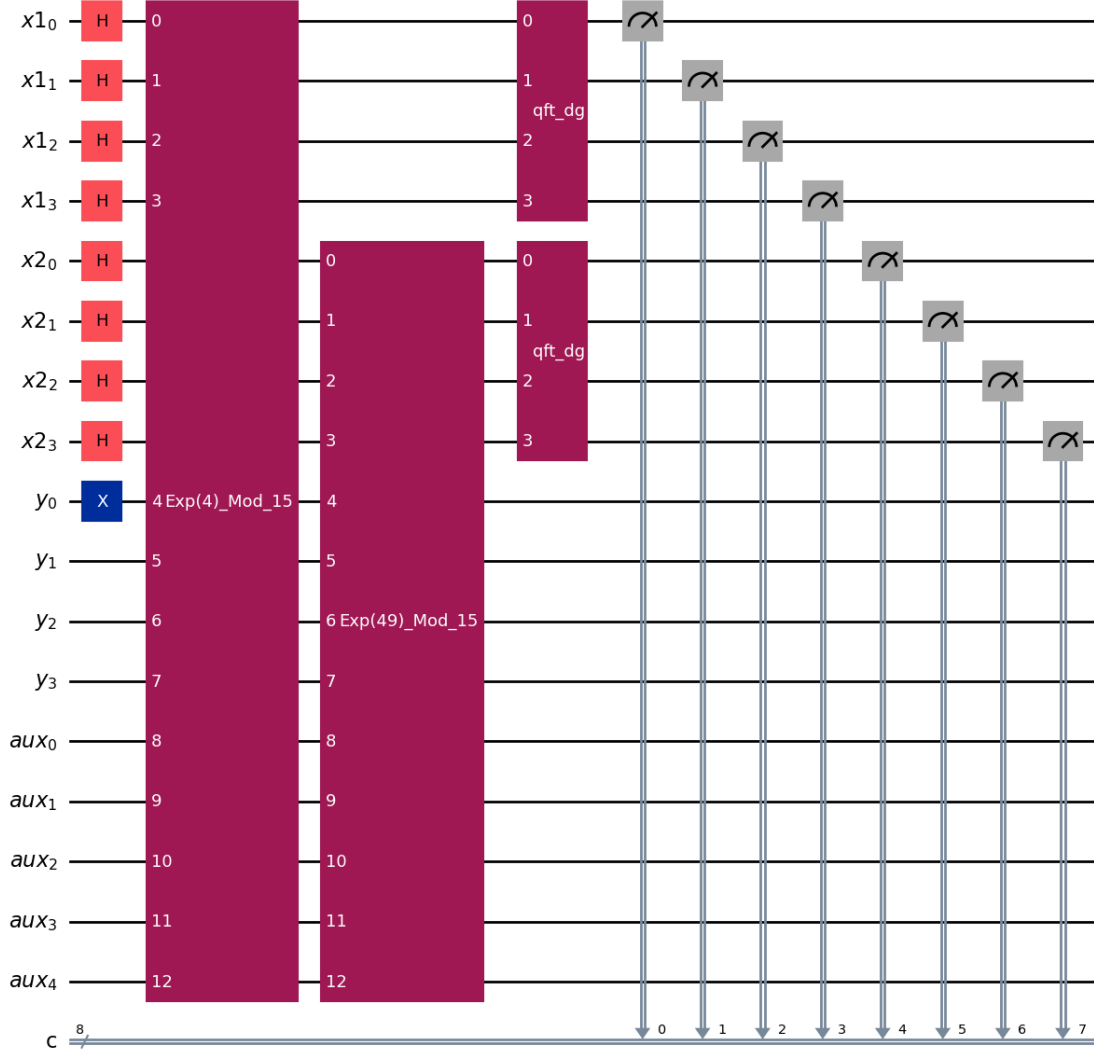


Figure 1: Constructed $N = 15$ circuit. Two four-qubit exponent registers control modular-exponentiation blocks acting on a shared four-qubit output register and five-qubit auxiliary register. Separate inverse QFT gates are followed by measurement of all eight exponent qubits.

7 Classical Post-Processing Layers

7.1 Measurement-vector extraction

Qiskit renders classical bitstrings in a display order that does not directly match the logical order of the exponent-register list. The notebook therefore removes spaces, reverses the bitstring, partitions it into d chunks of n_d bits, reverses each chunk for binary conversion, and returns

$$\mathbf{z} = (z_1, \dots, z_d). \quad (14)$$

This conversion is essential: without it, a correct quantum distribution could be interpreted as the wrong multidimensional sample set.

7.2 Toy continued-fraction diagnostic

For each coordinate z_i , the diagnostic routine treats

$$\frac{z_i}{M} \approx \frac{s}{r} \quad (15)$$

as a Shor-like rational approximation and obtains a denominator candidate with `Fraction.limit_denominator`. It tests that denominator and its first five positive multiples. A candidate order r is accepted only if it produces a nontrivial factor through

$$\gcd(a^{r/2} - 1, N) \quad \text{or} \quad \gcd(a^{r/2} + 1, N), \quad (16)$$

with the standard requirements that r be even and $a^{r/2} \not\equiv \pm 1 \pmod{N}$.

This routine is deliberately labeled *toy post-processing*. Regev’s algorithm uses correlations among multiple high-dimensional samples and lattice-based reconstruction; independent continued fractions on individual coordinates do not reproduce that procedure.

7.3 Lattice interface

The placeholder reports N , d , n_d , the selected bases, and the leading measured vectors. Architecturally, this is useful because it fixes the data contract for the missing stage: a lattice routine will receive integer vectors, sampling probabilities or multiplicities, base metadata, and the Fourier modulus. The next implementation step is therefore localized rather than requiring a redesign of the quantum front end.

8 Experimental Environment and Methodology

The notebook was executed in a Google Colab-style environment using Qiskit 2.5.0, Qiskit Aer 0.17.2, and a cloned modular-arithmetic repository. The end-to-end experiments used:

- `AerSimulator`;
- 128 shots per simulated instance;
- transpiler optimization level 1;
- inverse QFT in the main input-driven pipeline;
- the `cover_2n` register-sizing mode.

Circuit statistics were collected both before and after transpilation. The pre-transpilation circuit treats modular exponentiation and QFTs as high-level composite operations; post-transpilation statistics expose the substantial cost of decomposing those blocks into the simulator’s basis operations.

9 Experimental Results

9.1 $N = 15$

For $N = 15$, the selected parameters were

$$n = 4, \quad d = 2, \quad n_d = 4, \quad M = 16, \quad (17)$$

with eight exponent qubits and 17 total logical qubits. The accepted bases were $[4, 49]$, while the base-selection GCD checks recorded $(3, 5)$ and $(5, 3)$ as early factors. Reduced modulo 15, both accepted bases equal four, so the two dimensions are not arithmetically independent.

Table 2: Circuit statistics for the $N = 15$ simulation.

Metric	Before transpilation	After transpilation
Qubits	17	17
Depth	5	9,242
Hadamard gates	8	16
Composite modular exponentiation	2	0
Inverse-QFT blocks	2	0
CCX gates	0	6,002
CX gates	0	2,944
Controlled swaps	0	32
Controlled phase gates	0	12
Measurements	8	8

The logical depth of five expands by a factor of approximately 1.85×10^3 after decomposition. The complete observed distribution over 128 shots was:

Table 3: Measured Regev-style vectors for $N = 15$.

Vector	Count	Probability
$(8, 8)$	69	0.5391
$(0, 0)$	59	0.4609

The distribution is sharply concentrated on two points. This is consistent with the strong degeneracy introduced by the small modulus and duplicate reduced bases; it is a circuit-level observation rather than evidence of general Regev sampling behavior.

9.2 $N = 21$

For $N = 21$,

$$n = 5, \quad d = 3, \quad n_d = 4, \quad M = 16, \quad (18)$$

producing 12 exponent qubits and 23 logical qubits. The accepted bases were $[4, 25, 121]$, equivalent to $[4, 4, 16] \pmod{21}$, and the GCD checks encountered the factors three and seven.

Table 4: Circuit statistics for the $N = 21$ simulation.

Metric	Before transpilation	After transpilation
Qubits	23	23
Depth	6	22,586
Hadamard gates	12	24
Composite modular exponentiation	3	0
Inverse-QFT blocks	3	0
CCX gates	0	15,314
CX gates	0	6,216
Controlled swaps	0	60
Controlled phase gates	0	18
Measurements	12	12

The decomposition increases depth by approximately 3.76×10^3 . The dominant measured vectors were:

Table 5: Leading measured vectors for $N = 21$; 128 total shots were executed.

Vector	Count	Probability
(0, 0, 0)	37	0.2891
(11, 11, 5)	20	0.1563
(5, 5, 11)	11	0.0859
(6, 5, 11)	7	0.0547
(5, 6, 11)	6	0.0469
(5, 5, 10)	6	0.0469
(5, 6, 10)	3	0.0234
(11, 10, 5)	3	0.0234

Unlike the $N = 15$ result, the distribution contains several clustered neighboring vectors around (5, 5, 11) and (11, 11, 5). The zero vector remains the single most frequent outcome. A rigorous interpretation of these correlations requires the missing multidimensional reconstruction step.

9.3 $N = 57$ arithmetic-construction test

A separate exploratory cell constructs three modular-exponentiation blocks for $N = 57$. With $n = 6$ and $d = 3$, the 12 exponent qubits are split into three four-qubit registers, together with a six-qubit output register and seven-qubit auxiliary register. This gives 25 logical qubits.

During base generation, the candidate prime three divides 57, and the code reports $57 = 3 \times 19$ before the circuit is executed. The accepted squared-prime bases used for the displayed gates are 4, 25, and 49. This cell verifies the compatibility of the revised exponent-register width with the modular-exponentiation interface; it is not a measurement-based factorization experiment.

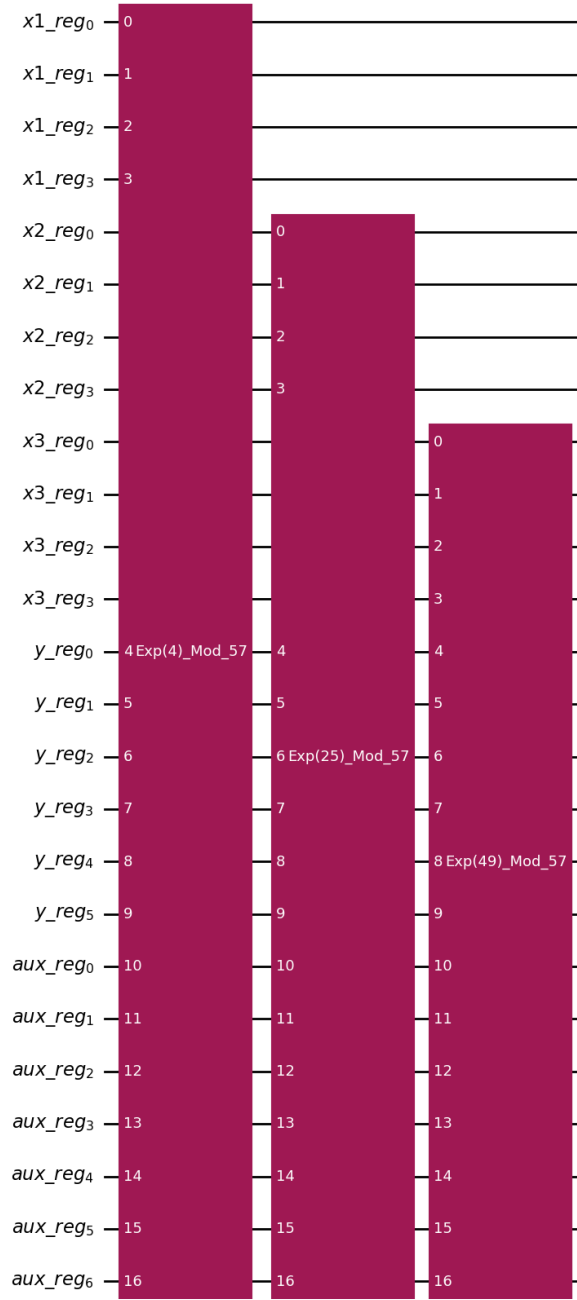


Figure 2: Exploratory $N = 57$ construction showing three four-qubit exponent registers controlling modular-exponentiation blocks on a shared six-qubit output register and seven-qubit auxiliary register.

9.4 $N = 2021$ construction and resource estimate

For $N = 2021$, the notebook computes

$$n = 11, \quad d = 4, \quad n_d = 6, \quad M = 64. \quad (19)$$

The design therefore uses 24 exponent qubits, 11 output qubits, 12 auxiliary qubits, and 47 logical qubits total. Four modular-exponentiation blocks are constructed using the squared-prime bases 4, 9, 25, and 49. The circuit is drawn but not transpiled or simulated, which is an appropriate boundary given the much larger arithmetic decomposition.

The exploratory cell measures `range(2*n)`, or 22 exponent qubits, even though $dn_d = 24$. This leaves two exponent qubits unmeasured. The later generalized circuit builder corrects the issue by constructing an explicit list of every qubit in every exponent register and measuring all dn_d qubits.

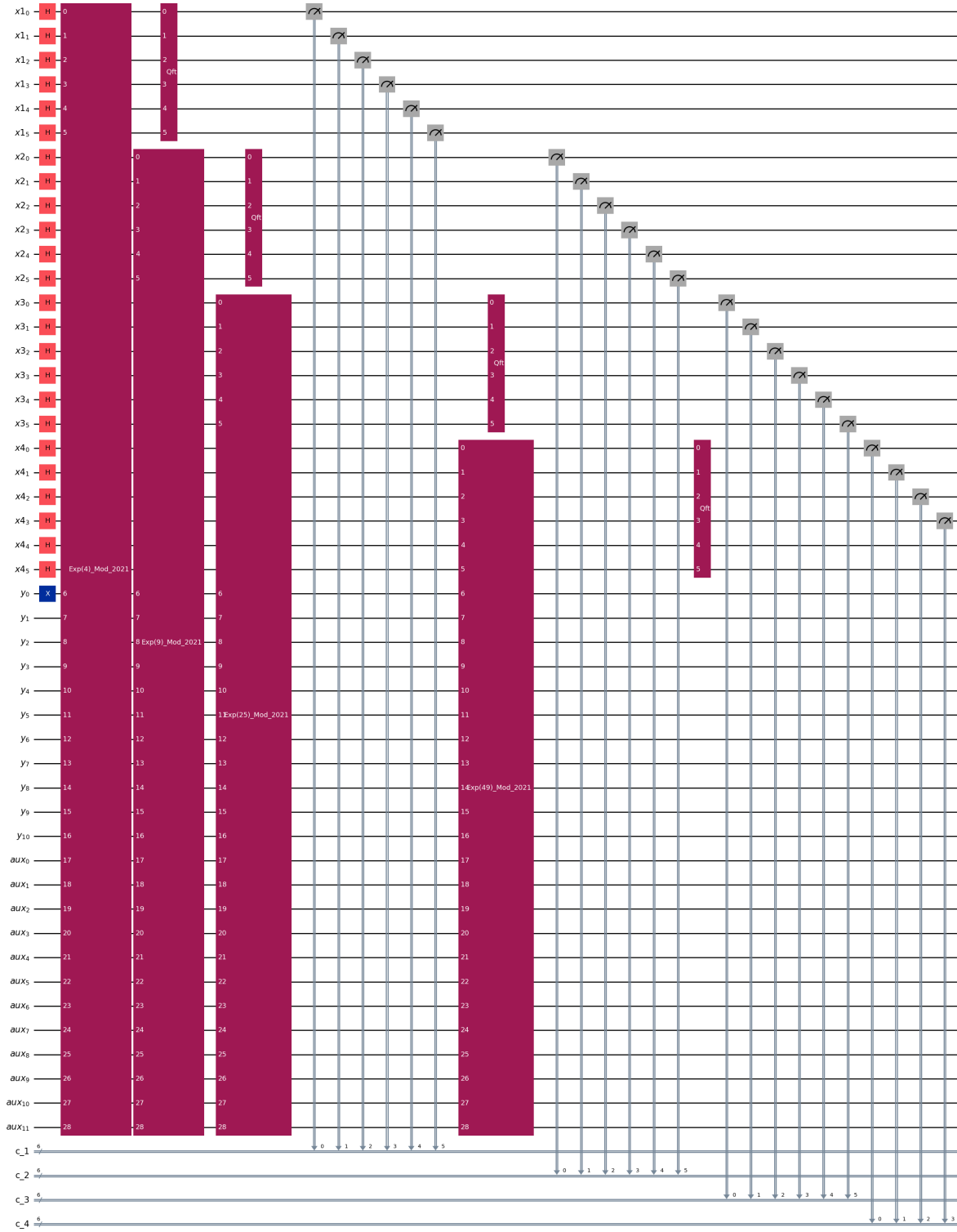


Figure 3: Exploratory $N = 2021$ circuit construction. The figure demonstrates four-dimensional register allocation and shared modular work registers, while also exposing the incomplete 22-of-24 measurement mapping in this earlier cell.

9.5 Logical resource scaling

Table 6: Register-level resource estimates generated without large-instance simulation.

N	n	d	n_d	dn_d	Q_{logical}
15	4	2	4	8	17
21	5	3	4	12	23
57	6	3	4	12	25
2021	11	4	6	24	47

The table shows moderate register growth at these toy sizes, but the transpilation results demonstrate that logical-qubit counts alone substantially understate execution difficulty. Reversible modular arithmetic dominates the decomposed depth and multi-qubit gate count.

10 Scientific Interpretation

10.1 What has been demonstrated

The notebook successfully demonstrates that:

- a single parameterized code path can generate multidimensional factoring circuits from arbitrary small odd composite inputs;
- the exponent width can be distributed across d registers while preserving the modular-exponentiation interface;
- sequential exponentiation blocks can share a common modular output and auxiliary workspace;
- the product inverse QFT can be applied register-by-register;
- all exponent measurements can be decoded into ordered integer vectors;
- simulator execution produces nonuniform, structured multidimensional distributions;
- decomposition of apparently shallow high-level circuits produces very deep arithmetic circuits;
- the classical reconstruction stage can be cleanly attached through a defined sample interface.

10.2 What has not yet been demonstrated

The current results do not establish:

- successful factor recovery using Regev’s lattice post-processing;
- asymptotic gate savings relative to an equivalently optimized Shor implementation;
- resilience under realistic device noise;
- feasibility on present-day hardware;
- arithmetic correctness for every computational-basis input through systematic unit tests;
- statistical convergence of the measured distribution across repeated independent circuit runs.

This distinction is especially important because Regev’s theoretical construction requires repeated quantum samples and a nontrivial classical reconstruction whose correctness rests on a number-theoretic heuristic assumption [1]. Subsequent work has studied both implementation behavior and space/noise refinements [2, 3], emphasizing that practical performance depends strongly on arithmetic design, sample quality, and post-processing.

11 Limitations and Technical Risks

11.1 Missing lattice reconstruction

The central limitation is the absence of the actual lattice-reduction and relation-recovery procedure. The placeholder is architecturally useful, but a complete experiment requires construction of the relevant lattice from multiple samples, an LLL- or BKZ-based reduction step, extraction of short relations, and a verified conversion from those relations to a factor of N .

11.2 Toy post-processing is not Regev post-processing

The continued-fraction routine treats each coordinate as if it independently approximated a one-dimensional phase. This can be a useful sanity check for tiny circuits, but it discards the high-dimensional structure that motivates the algorithm.

11.3 Classical factor leakage during base generation

The GCD checks identify factors for all three small composites tested in the notebook. The code should retain this early exit for correctness, but experiments intended to test sample-based recovery should use inputs and base-selection policies for which the factors are not encountered before circuit execution. Results should separately label *GCD shortcut*, *toy order recovery*, and *lattice recovery*.

11.4 Base collisions modulo N

Squared primes can map to identical residues: $4 \equiv 49 \pmod{15}$ and $4 \equiv 25 \pmod{21}$. Duplicate or strongly related bases reduce effective dimensional diversity. A stronger generator should test uniqueness modulo N , subgroup structure, and possibly multiplicative independence criteria appropriate to the selected Regev variant.

11.5 Forward/inverse QFT inconsistency

The initial `multidim_qft` helper applies a forward QFT, while the main simulation builder defaults to the inverse QFT. The $N = 2021$ exploratory circuit uses the earlier forward helper. The codebase should expose one clearly named convention, document the expected sign, and include a phase-sensitive unit test.

11.6 Measurement inconsistency in the exploratory large circuit

The $N = 2021$ cell measures only $2n = 22$ of its 24 exponent qubits. The generalized builder fixes this, but the old cell should either be removed or revised to avoid contradictory examples.

11.7 Resource estimate boundaries

Equation (7) counts logical wires in the present circuit layout. It does not include routing overhead, connectivity constraints, error correction, magic-state production, or physical fault-tolerance costs. Moreover, gate counts depend on the chosen arithmetic library and transpilation target.

11.8 Simulation and validation coverage

Only two complete Aer simulations are preserved, both with 128 shots and no explicit noise model. The notebook does not yet include automated tests that compare the modular arithmetic against a classical oracle for every basis state at small n , nor does it test ancilla cleanup by statevector or unitary equivalence.

12 Recommended Next Research Phase

A rigorous next phase should proceed in the following order.

12.1 Phase I: arithmetic verification

1. For small N , exhaustively test each controlled multiplication and modular exponentiation gate on all valid computational-basis inputs.
2. Verify that the auxiliary register returns to $|0\rangle$ and that the intended map is unitary on the full encoded space.
3. Add regression tests for exponent widths not equal to $2n$.
4. Compare gate counts across arithmetic implementations rather than treating one cloned construction as fixed.

12.2 Phase II: sampling verification

1. Enforce unique accepted bases modulo N .
2. Standardize the inverse-QFT convention.
3. Run repeated independent circuit batches, not only a single 128-shot execution.
4. Quantify distribution stability with total variation distance or confidence intervals.
5. Compare exact and approximate QFT truncation.

12.3 Phase III: full lattice post-processing

1. Implement the sample-to-lattice embedding from the selected Regev formulation.
2. Use a reproducible lattice library and record reduction parameters.
3. Validate short-vector recovery on synthetically generated noiseless samples before connecting it to circuit output.
4. Track the number of independent circuit executions required for successful factor recovery.
5. Distinguish failures due to sampling, base choice, lattice reduction, and final number-theoretic conversion.

12.4 Phase IV: comparative and noise-aware study

1. Build a Shor baseline with the same modular-arithmetic backend and transpilation target.
2. Compare logical qubits, decomposed two- and three-qubit gates, depth, runtime, and factor-recovery probability.
3. Introduce controlled depolarizing, readout, and coherent-error models.
4. Explore the noise-tolerant sample filtering proposed in later Regev-based work [3].

13 Conclusion

The notebook represents a substantial implementation step beyond an isolated QFT or textbook order-finding demonstration. It contains a coherent reversible-arithmetic stack, a parameterized multidimensional circuit generator, simulator execution, resource reporting, robust measurement decoding, and a clear boundary between quantum sampling and classical reconstruction. The experiments for $N = 15$ and $N = 21$ confirm that the constructed circuits transpile and produce structured multidimensional outcomes, while the $N = 57$ and $N = 2021$ cells probe register generalization and scaling.

The most important scientific conclusion is not that Regev factorization has already been completed, but that the quantum sampling infrastructure has been built far enough to expose the real remaining research problem. The next decisive contribution is a mathematically faithful lattice post-processing implementation accompanied by arithmetic unit tests, nondegenerate base selection, repeated-sample analysis, and direct comparison against a controlled Shor baseline. With those additions, the notebook can evolve from a strong implementation scaffold into a defensible end-to-end experimental study.

A Representative Core Logic

The following condensed pseudocode captures the main circuit-construction logic without reproducing the full notebook.

Listing 1: Condensed input-driven circuit construction.

```

1 def build_regev_circuit(N):
2     n = N.bit_length()
3     d = ceil(sqrt(n))
4     nd = ceil(2 * n / d)
5
6     x = [QuantumRegister(nd) for _ in range(d)]
7     y = QuantumRegister(n)
8     aux = QuantumRegister(n + 1)
9     c = ClassicalRegister(d * nd)
10    qc = QuantumCircuit(*x, y, aux, c)
11
12    for x_i in x:
13        qc.h(x_i)
14        qc.x(y[0])
15
16    bases = generate_coprime_squared_prime_bases(N, d)
17    for i in range(d):
18        gate = modular_exponentiation_gate(bases[i], N, n, nd)

```

```

19     qc.append(gate, list(x[i]) + list(y) + list(aux))
20
21     for x_i in x:
22         qc.append(QFTGate(nd).inverse(), list(x_i))
23
24     all_x_qubits = [q for x_i in x for q in x_i]
25     qc.measure(all_x_qubits, list(c))
26     return qc

```

B Reproducibility Checklist

- Qiskit version printed by the notebook: 2.5.0.
- Aer version installed in the notebook environment: 0.17.2.
- Simulator: AerSimulator.
- Transpiler optimization level: 1.
- Shots for $N = 15$ and $N = 21$: 128 each.
- Main QFT convention: inverse QFT on each dimension.
- Main sizing convention: $n_d = \lceil 2n/d \rceil$.
- Arithmetic dependency: cloned `regev-quantum-algorithm` repository, especially the Häner-style modular-adder components and circuit-construction utilities.
- Preserved output artifacts: circuit diagrams for $N = 15$, $N = 57$, and $N = 2021$.

References

- [1] O. Regev, “An Efficient Quantum Factoring Algorithm,” *arXiv preprint arXiv:2308.06572*, 2023. <https://arxiv.org/abs/2308.06572>
- [2] P. Pawlitko, N. Moćko, M. Niemiec, and P. Cholda, “Implementation and Analysis of Regev’s Quantum Factorization Algorithm,” *arXiv preprint arXiv:2502.09772*, 2025. <https://arxiv.org/abs/2502.09772>
- [3] S. Ragavan and V. Vaikuntanathan, “Space-Efficient and Noise-Robust Quantum Factoring,” *arXiv preprint arXiv:2310.00899*, 2023. <https://arxiv.org/abs/2310.00899>